САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

ОТЧЁТ

по лабораторной работе № 2

Выполнил:
Цатинян Артём
Группа K3439

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Цель работы

Подключить сайт бронирования столиков tablely, созданный в первой лабораторной работе, к имитации REST API. Реализовать получение данных и изменение бронирований средствами JavaScript и fetch, а также авторизацию пользователя.

Постановка задачи

Необходимо заменить демонстрационную работу с данными на взаимодействие с JSON-сервером. Каталог ресторанов должен формироваться по ответу сервера; зарегистрированный пользователь должен иметь возможность создавать, просматривать, изменять и отменять собственные бронирования.

Ход работы

1. Подготовка проекта

В отдельной ветке от первой лабораторной работы создан каталог lab2. При разработке сохранены страницы и стили первого проекта: каталог, вход, регистрация, страница ресторана и личный кабинет. Клиентский JavaScript разделён по назначению на модули взаимодействия с API, авторизации, каталога, страницы ресторана, личного кабинета и общих функций интерфейса.

Для имитации сервера использован json-server. Начальные данные находятся в seed.json: при первом запуске на их основе создаётся рабочий db.json. В нём хранятся рестораны, зарегистрированные пользователи и бронирования. Рабочий файл исключён из Git, чтобы локальные пользовательские данные не попадали в репозиторий.

2. Маршруты API

Сервер предоставляет публичный каталог ресторанов и защищённые операции для пользователя и его броней. Запросы отправляются через fetch, а ответы обрабатываются в общем модуле API. При ошибке сети или ответа сервера пользователю показывается сообщение.

Использованные маршруты:

• GET /api/restaurants — получение и фильтрация каталога;
• GET /api/restaurants/:id — получение данных ресторана;
• POST /api/register и POST /api/login — регистрация и вход;
• GET /api/me и POST /api/logout — получение текущего пользователя и выход;
• GET /api/bookings — список броней текущего пользователя;
• POST /api/bookings — создание брони;

• PATCH /api/bookings/:id — изменение брони;

• DELETE /api/bookings/:id — отмена брони.

3. Авторизация и доступ к данным

При регистрации сервер проверяет имя, адрес электронной почты и длину пароля. Перед сохранением пароль преобразуется с помощью scrypt и индивидуальной соли. При входе сервер проверяет пароль и выдаёт случайный токен. Клиент сохраняет токен на время вкладки и передаёт его в заголовке Authorization последующих запросов.

Сервер связывает токен с пользователем и проверяет принадлежность брони перед чтением, изменением или удалением. Сессии хранятся в памяти сервера и сбрасываются при его перезапуске: это ограничение демонстрационного решения.

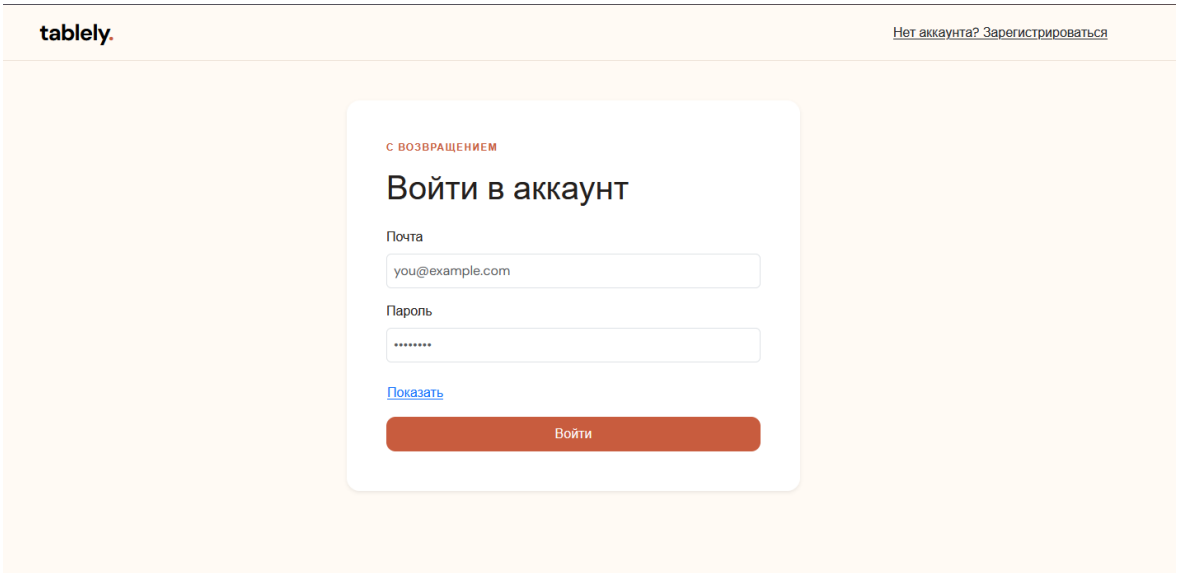


Рисунок 1 — Форма входа в аккаунт

4. Каталог ресторанов

При открытии главной страницы клиент запрашивает список ресторанов, а затем формирует карточки по данным ответа. Выбор кухни и района изменяет параметры запроса к серверу; фильтры работают одновременно. При отсутствии подходящих результатов отображается соответствующее сообщение.

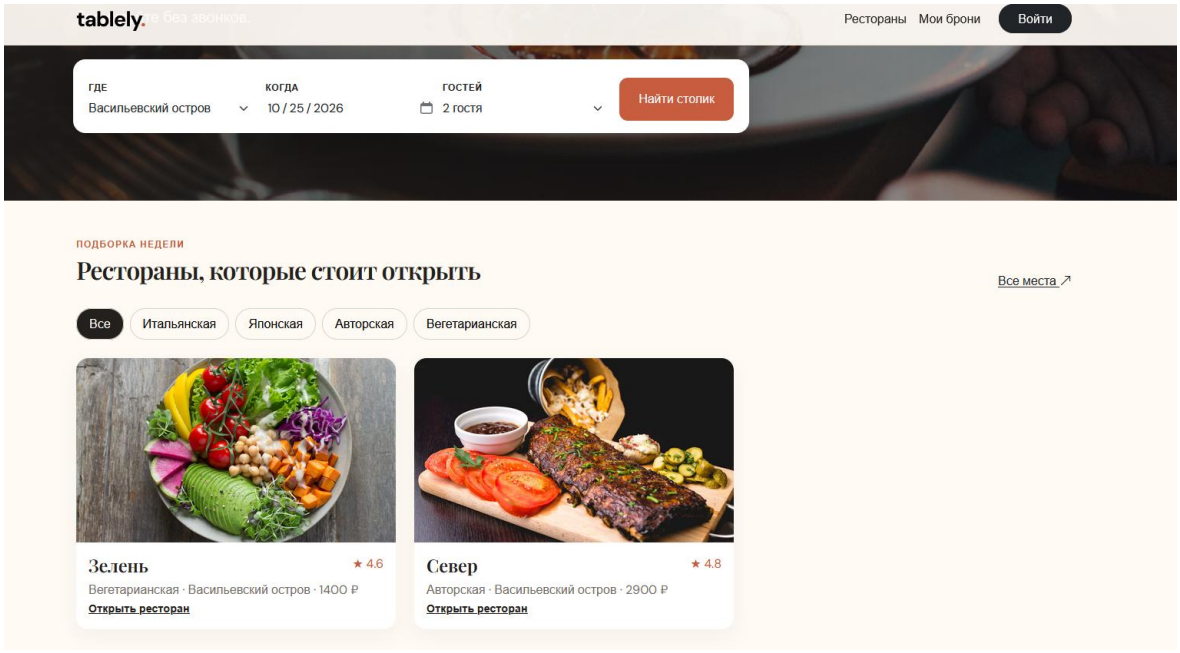


Рисунок 2 — Каталог с фильтрацией по району

5. Бронирование и личный кабинет

На странице ресторана отображаются описание, адрес, средний чек, блюдо и рейтинг, полученные по идентификатору ресторана. В форме выбираются дата, время и количество гостей. При отправке создаётся запрос POST; сервер проверяет дату, допустимые значения и наличие другой брони этого пользователя на то же время.

В личном кабинете пользователь видит свои бронирования. Для предстоящей брони доступны изменение и отмена: данные редактируемой записи загружаются в форму, затем сохраняются запросом PATCH; отмена выполняется запросом DELETE.

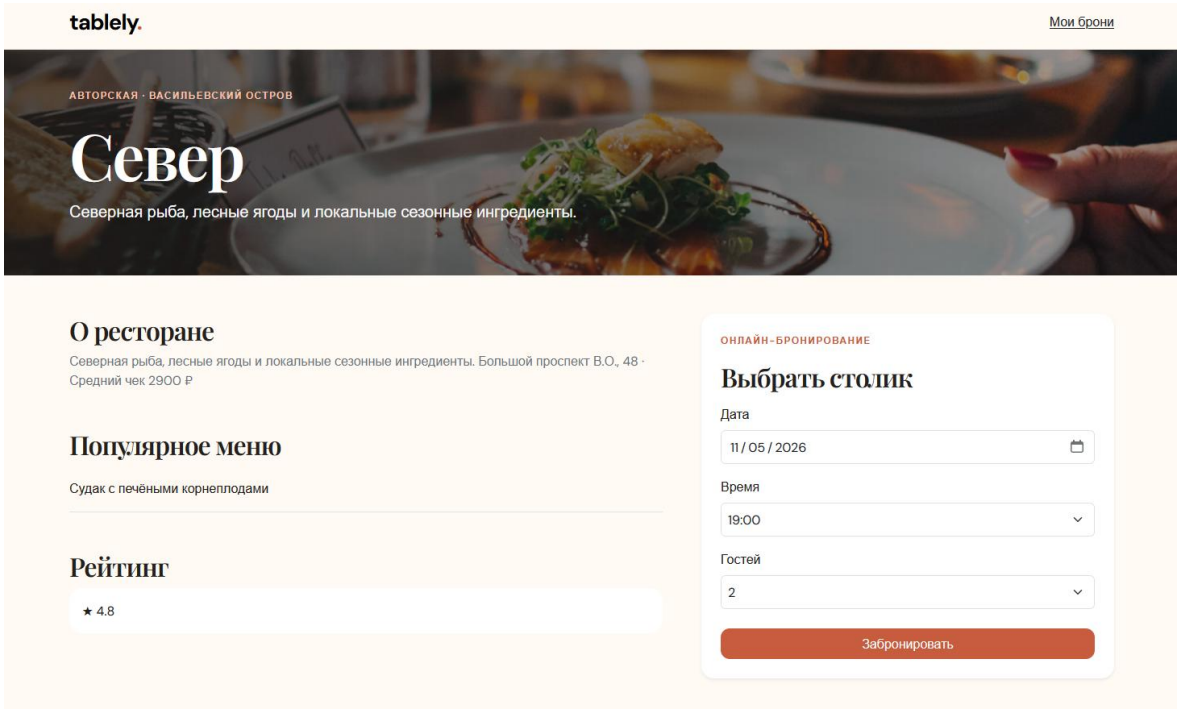


Рисунок 3 — Страница ресторана и форма бронирования

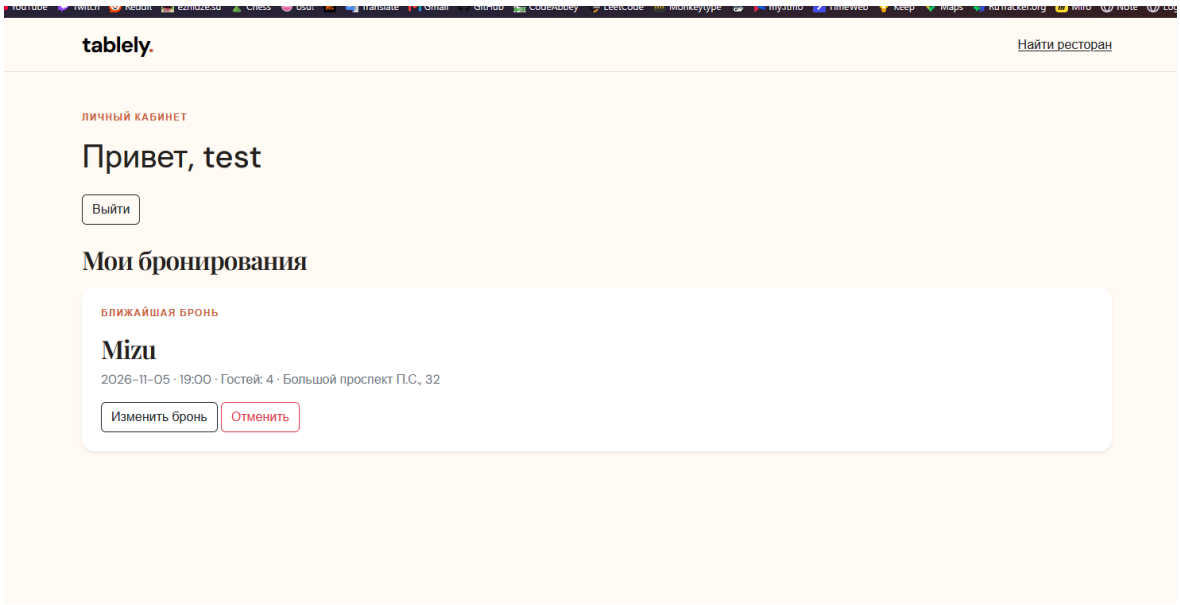


Рисунок 4 — Бронирование в личном кабинете

6. Проверка работы

Интеграционный тест API проверяет регистрацию, повторную регистрацию, вход с верным и неверным паролем, фильтрацию, создание и изменение брони, сохранение в файл, конфликт времени, отмену и выход. Отдельно проверяется, что другой пользователь не может получить или удалить чужую бронь.

Браузерный сценарий проверяет фильтрацию каталога, регистрацию, создание, редактирование и отмену бронирования, а также отсутствие горизонтального переполнения на мобильной ширине.

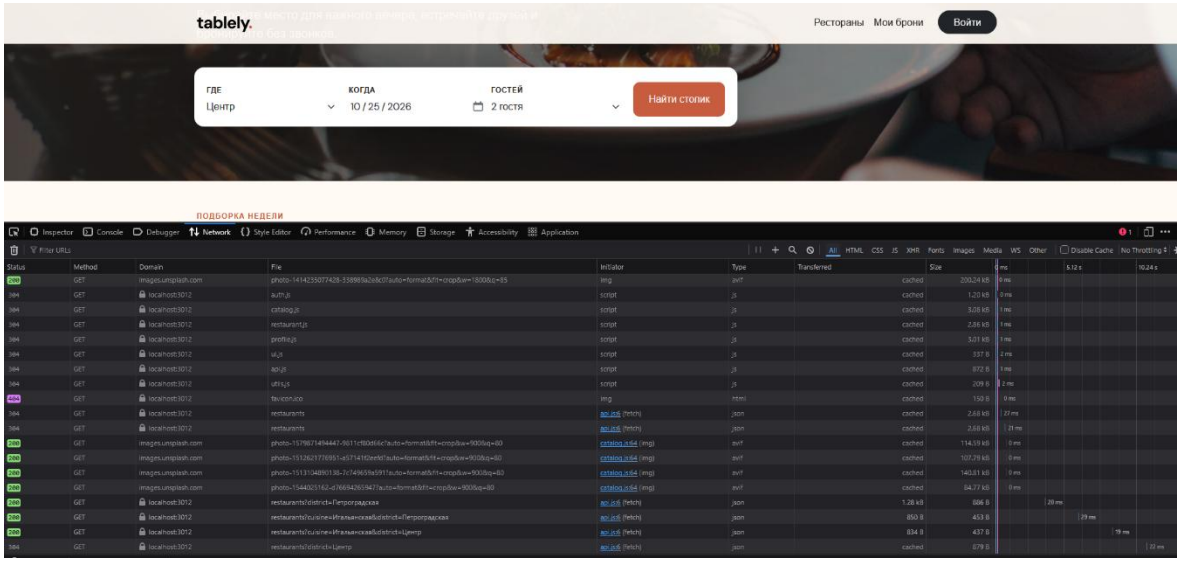


Рисунок 5 — Запросы к каталогу в инструментах разработчика

Вывод

В ходе работы интерфейс tablely был подключён к имитации REST API на базе json-server. Получение каталога, авторизация и управление бронированиями выполняются асинхронными запросами; сервер проверяет права пользователя и сохраняет изменения в JSON-файл. Получен практический опыт работы с fetch, HTTP-методами, обработкой ошибок и разделением клиентского и серверного кода.